

All Business Questions with Queries and Output

01_top_5_movies.sql

Business Question:

Identify the top 5 highest-rated movies in the database.

Purpose:

This analysis helps determine the best-performing movies based on user ratings. It can be useful for creating recommendation lists, highlighting popular content, or identifying high-quality movies for promotional campaigns.

Query:

```
select m.title,r.rating
from ratings r
join movies m
on m.movie_id=r.movie_id
order by rating DESC
limit 5;
```

Output:

Data Output

Messages

Notifications

≡+

▼

▼

SQL

	title text	rating numeric (2,1)
1	The Shawshank Redempti...	9.8
2	3 Idiots	9.5
3	Taare Zameen Par	9.4
4	Interstellar	9.3
5	Dangal	9.1

02_movies_above_9_india.sql

Business Question:

Which movies have received ratings above 9 from users located in India?

Purpose:

This query identifies movies that are highly appreciated by Indian users, allowing businesses to understand regional preferences and recommend content based on geographic audience behavior.

Query:

```
select r.rating , m.title , u.user_id
from ratings r
join movies m
on m.movie_id=r.movie_id
join users u
on u.user_id=r.user_id
where r.rating>=9 and u.country='India';
```

Output:

Data Output Messages Notifications			
	rating numeric (2,1) 🔒	title text 🔒	user_id integer 🔒
1	9.0	Inception	1
2	9.0	The Dark Knight	1
3	9.3	Interstellar	5
4	9.4	Taare Zameen P...	1

03_movies_per_genre.sql

Business Question:

How many movies belong to each genre?

Purpose:

This analysis provides an overview of the movie catalog distribution. It helps identify which genres have the largest or smallest collections, supporting content planning and inventory analysis.

Query:

```
select g.genre_name, count(m.movie_id) as TotalMovies
from genres g
join movie_genres mg
on g.genre_id=mg.genre_id
join movies m
on m.movie_id=mg.movie_id
group by g.genre_name;
```

Output:

Data Output Messages Notifications			
	genre_name	totalmovies	
1	Biography	1	
2	Thriller	3	
3	Romance	1	
4	Comedy	2	
5	Crime	2	
6	Drama	7	
7	Action	4	

04_actor_with_most_movies.sql

Business Question:

Which actor has appeared in the highest number of movies?

Purpose:

This query identifies the most frequently featured actor in the database. It can be useful for talent analysis, popularity tracking, and actor-based recommendation systems.

Query:

```
select a.name, count(movie_id)as TotalMovies
from actors a
join movie_actors ma
on a.actor_id=ma.actor_id
group by a.name
order by Totalmovies desc
limit 1;
```

Output:

Data Output			Messages	Notifications				
								
	name text		totalmovies bigint					
1	Aamir Kh...		4					

05_average_rating_per_user.sql

Business Question:

What is the average rating given by each user?

Purpose:

This analysis evaluates user rating behavior by calculating each user's average rating. It helps identify users who generally rate movies strictly or generously.

Query:

```
select u.name , u.user_id,round(avg(r.rating),2)as Avg_rating
from ratings r
join users u on u.user_id=r.user_id
group by u.user_id;
```

Output:

Data Output Messages Notifications			
		user_id [PK] integer	avg_rating numeric
1		4	8.95
2	Sop...	2	9.20
3	Carl...	3	9.15
4	Priya	5	9.30
5	Aarav	1	9.13

06_release_rank.sql

Business Question:

Assign a release rank to each movie based on its release year.

Purpose:

This query ranks movies chronologically using a window function.

It helps analyze release trends and demonstrates the use of SQL ranking functions.

Query:

```
select title,release_year,
```

```
RANK() OVER(order by release_year) as Release_rank
```

```
from movies;
```

Output:

Data Output				Messages	Notifications
	title text	release_year integer	release_rank bigint		
1	12 Angry Men	1957	1		
2	The Godfather	1972	2		
3	Sholay	1975	3		
4	Pulp Fiction	1994	4		
5	The Shawshank Redemp...	1994	4		
6	Forrest Gump	1994	4		
7	Titanic	1997	7		
8	Fight Club	1999	8		
9	The Matrix	1999	8		
10	Lagaan	2001	10		
11	Dil Chahta Hai	2001	10		
12	Munna Bhai M.B.B.S.	2003	12		
13	Swades	2004	13		
14	Black	2005	14		
15	Batman Begins	2005	14		
16	Don	2006	16		
17	The Pursuit of Happyness	2006	16		
18	Taare Zameen Par	2007	18		
19	Jab We Met	2007	18		
20	Chak De! India	2007	18		
21	Slumdog Millionaire	2008	21		

07_movie_classification.sql

Business Question:

Classify each movie as 'Classic', 'Modern', or 'New' based on its release year.

Purpose:

This categorization groups movies into different eras, making it easier to analyze viewing trends and organize content for recommendation or reporting purposes.

Query:

```
select title, release_year ,
CASE
    when release_year < 2000 THEN 'CLASSIC'
    when release_year > 2000 and release_year < 2020 then 'MODERN'
    else 'NEW'
end as Movie_Type
from movies;
```

Output:

	title text	release_year integer	movie_type text
1	Inception	2010	MODERN
2	3 Idiots	2009	MODERN
3	The Shawshank Redemp...	1994	CLASSIC
4	Dangal	2016	MODERN
5	The Dark Knight	2008	MODERN
6	Lagaan	2001	MODERN
7	Pulp Fiction	1994	CLASSIC
8	Swades	2004	MODERN
9	Interstellar	2014	MODERN
10	Taare Zameen Par	2007	MODERN
11	The Godfather	1972	CLASSIC
12	PK	2014	MODERN
13	Zindagi Na Milegi Dobara	2011	MODERN
14	Forrest Gump	1994	CLASSIC
15	Barfi!	2012	MODERN
16	The Matrix	1999	CLASSIC
17	Black	2005	MODERN
18	The Pursuit of Happyness	2006	MODERN
19	Dil Chahta Hai	2001	MODERN
20	The Social Network	2010	MODERN
21	Chak De! India	2007	MODERN
Total rows: 50		Query complete 00:00:00.210	

08_recommended_movie.sql

Business Question:

Recommend the highest-rated movie available in the database.

Purpose:

This query selects the top-rated movie as the "Movie of the Day."

It demonstrates how SQL can be used to generate simple recommendation systems based on rating data.

Query:

```
select m.title as Name , m.release_year as Release_Year , g.genre_name as Genre,  
a.name as Actor, r.rating as rating
```

```
from movies m
```

```
join movie_genres mg on m.movie_id=mg.movie_id
```

```
join genres g on mg.genre_id=g.genre_id
```

```
join movie_actors ma on m.movie_id=ma.movie_id
```

```
join actors a on a.actor_id=ma.actor_id
```

```
join ratings r on r.movie_id=m.movie_id
```

```
where r.rating>9
```

```
order by random()
```

```
limit 1 ;
```

Output:

Data Output Messages Notifications					
	name text	release_year integer	genre text	actor text	rating numeric (2,1)
1	The Shawshank Redempti...	1994	Crime	Morgan Freem...	9.8